

Fryktløs (GenAI) koding

Christian Chavez



Offentlig Fagdag — 23. april 2026

Kort om meg	2
Motivasjon	6
Kompilatoren som trygger	11
FFI i fremmed land? Nix!	19
Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!	25
Avslutning	31

Hvem er jeg?



- GenAI? Februar 2026!
- Nevroatypisk? Diagnostisert 2024
- Fingerbetennelse? Siden 2022
- Startet hos NAV & nais 2020!
- Avhengig av (smakfull) glutenfri mat
- Glad i:
 - Hiking
 - Sci-Fi
 - Engasjert i datakultur
 - Nix
 - Homelabbing





Nais

«En plattform laget av NAV for å gi fart og flyt til utviklerne av det offentlige Norge»





Hva tilbyr nais?

- *Utrulling*
kode til prod uten nedetid
- *Automatisering*
infra settes opp for deg
- *Full oversikt*
hva kjører, hva det koster
- *Innsikt*
verktøy for å forstå systemene
- *Sikkerhet*
solide defaults ut av boksen
- *Fellesskap*
mange bruker, mange bidrar

Hva er nais?

Brukes av

- NAV
- Statistisk sentralbyrå
- Landbruksdirektoratet
- Arbeidstilsynet

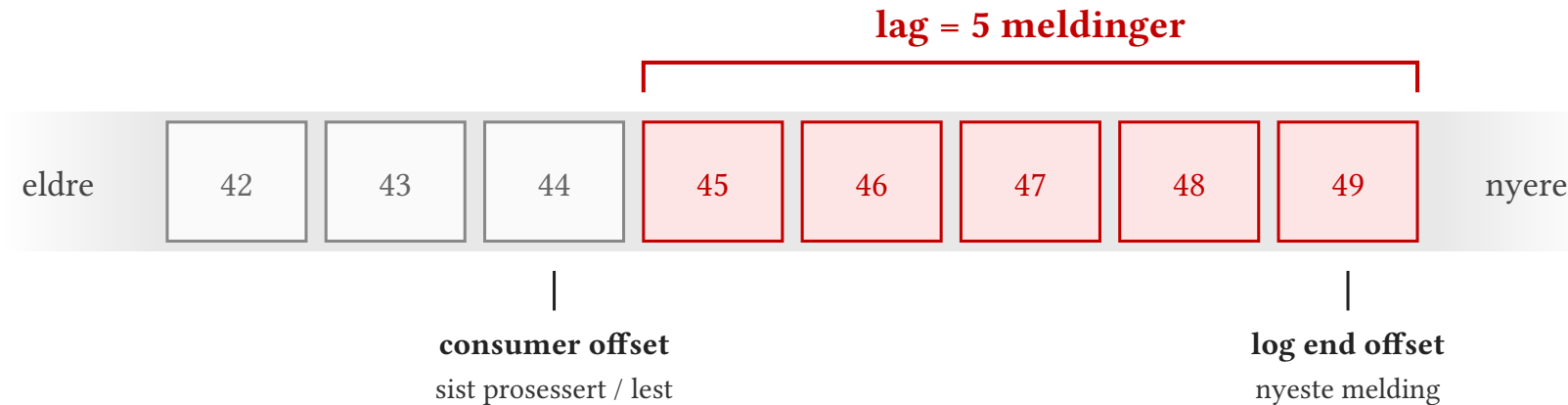
<https://nais.no>

Kort om meg	2
Motivasjon	6
Kompilatoren som trygger	11
FFI i fremmed land? Nix!	19
Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!	25
Avslutning	31

Vi trenger metrikker som informerer hvor langt **etterslep** tjenester som bruker **køer** henger.



→ trenger en pålitelig «lag-exporter»



Hva kan vi bruke lag-metrikker til?

HPA

automatisk lastskalering

Feilkøer

avslører meldinger tjenesten
feilet på

Varsling

team kan pinges før SLA brister

seglo/kafka-lag-exporter



- Scala (JVM)
 - siste commit jan. 2023, nå arkivert
- Andre JVM? ([lightbend/kafka-lag-exporter](#) — forløperen)
 - like forlatt
- Go? ([danielqsj/kafka_exporter](#), [linkedin/Burrow](#))
 - krevde metriknavn-re-labeling — drop-in var *viktigst*
 - slet med ytelse gitt vårt topic-/CG-volum



softwaremill/klag-exporter

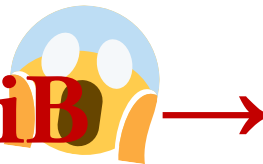


- Rust
 - godt skussmål ref ytelse
 - mer nylig vedlikeholdt enn mye annet
- Bruker [confluentinc/librdkafka](#)
 - vedlikeholdt av Kafka's utviklere
 - lang og lovende fartstid (~14 år)
 - fortsatt under utvikling

- **klag-exporter**
 - installert/testet på nais med liten kafkaklynge
 - ~5 topics
- nais har derimot kafkaklynger med **rundt omkring**
 - 1 429 topics
 - benyttet av over 772 apper
 - selv **klag-exporter** (rust) -> taklet dette dårlig



32+ GiB
tidlig forsøk



<1 GiB
etter (GenAI) refaktorering



Kort om meg	2
Motivasjon	6
Kompilatoren som trygger	11
FFI i fremmed land? Nix!	19
Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!	25
Avslutning	31

2. Første store grep

1. Rydd opp først

- cargo clippy
- «Lar meg bli kjent med koden»
- Null unwrap()s før AI slapp til



- Fra per-gruppe BaseConsumer (15–30 MB per)
 - til **én** delt AdminClient
- Angrepsvinkel lånt fra
 - seglo/kafka-lag-exporter
- Admin API manglet i rust-rdkafka
 - Forket fra j-santander/rust-rdkafka



220 linjer unsafe FFI¹ fjernet. Null unsafe blocks igjen.

$$O(\text{concurrent_groups} \times 25 \text{ MB}) \rightarrow O(1)$$



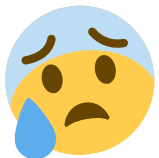
¹FFI (Foreign Function Interface): mekanismen for interop mellom språk — bindingen mot en ABI (f.eks. C-ABI).

Admin API endringen utgjorde **stor** forskjell. Vårt største cluster vokste fortsatt forbi **900 MiB** per pod.

```
2026-02-19 23:26:11+01:00: ~/Doc/nav/git/klag-exporter is 📦 v0.1.15 via 🚀 v1.92.0 w/nix-shell-env!❄️ * nav-dev:nais-system ◀ NAV
x10an14@nav-x10an14-t14 + > : kubectl get pods -l app=klag-exporter
NAME                                READY   STATUS    RESTARTS   AGE
klag-exporter-5b579bfdc6-n22fh      1/1     Running   0           24m

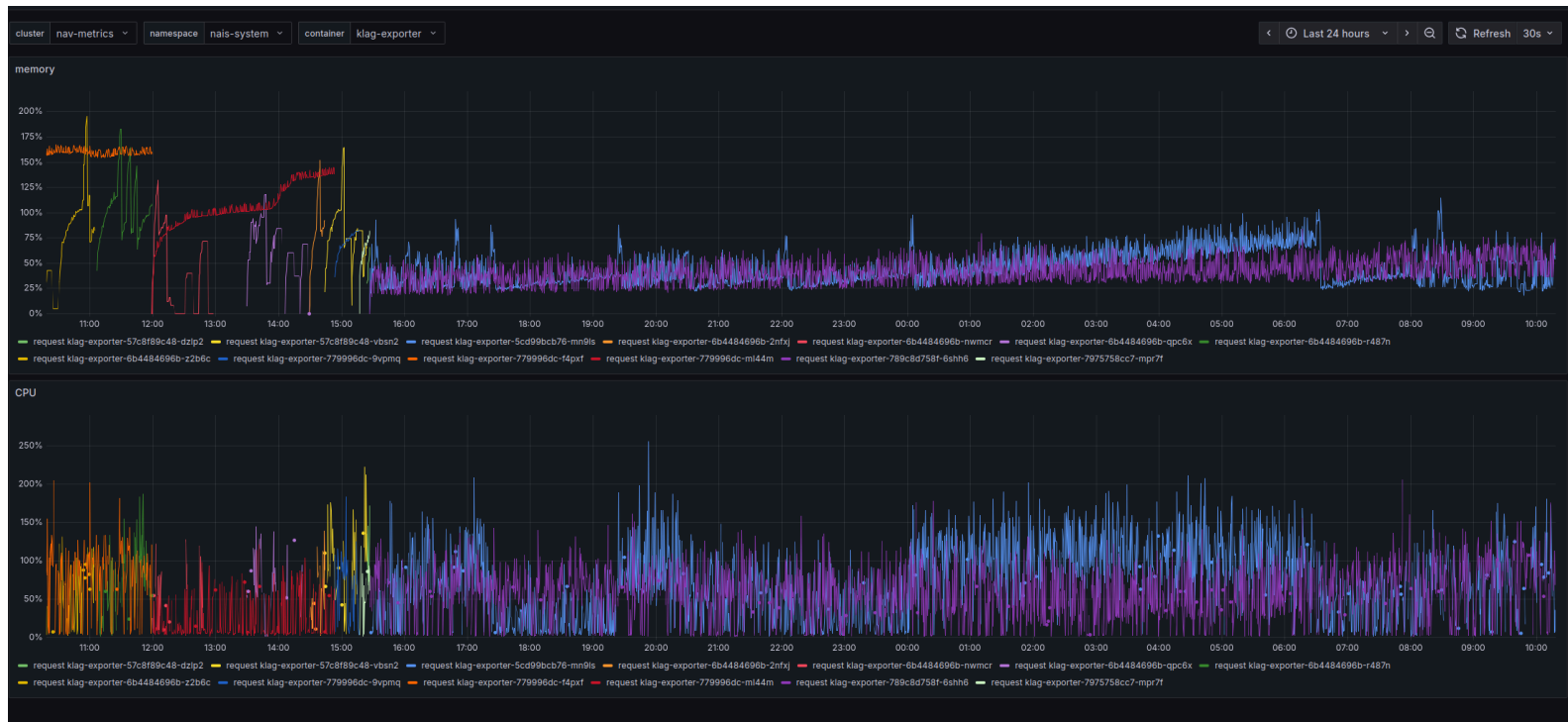
2026-02-19 23:33:02+01:00: ~/Doc/nav/git/klag-exporter is 📦 v0.1.15 via 🚀 v1.92.0 w/nix-shell-env!❄️ * nav-dev:nais-system ◀ NAV took 2s
x10an14@nav-x10an14-t14 > : kubectl top pod | grep lag-exporter
kafka-lag-exporter-55ff5d4877-4rfgc      452m      1471Mi
klag-exporter-5b579bfdc6-n22fh           236m      943Mi

2026-02-19 23:33:03+01:00: ~/Doc/nav/git/klag-exporter is 📦 v0.1.15 via 🚀 v1.92.0 w/nix-shell-env!❄️ * nav-dev:nais-system ◀ NAV
x10an14@nav-x10an14-t14 > : on 🗑️ deploy_our_own_helmchart
```



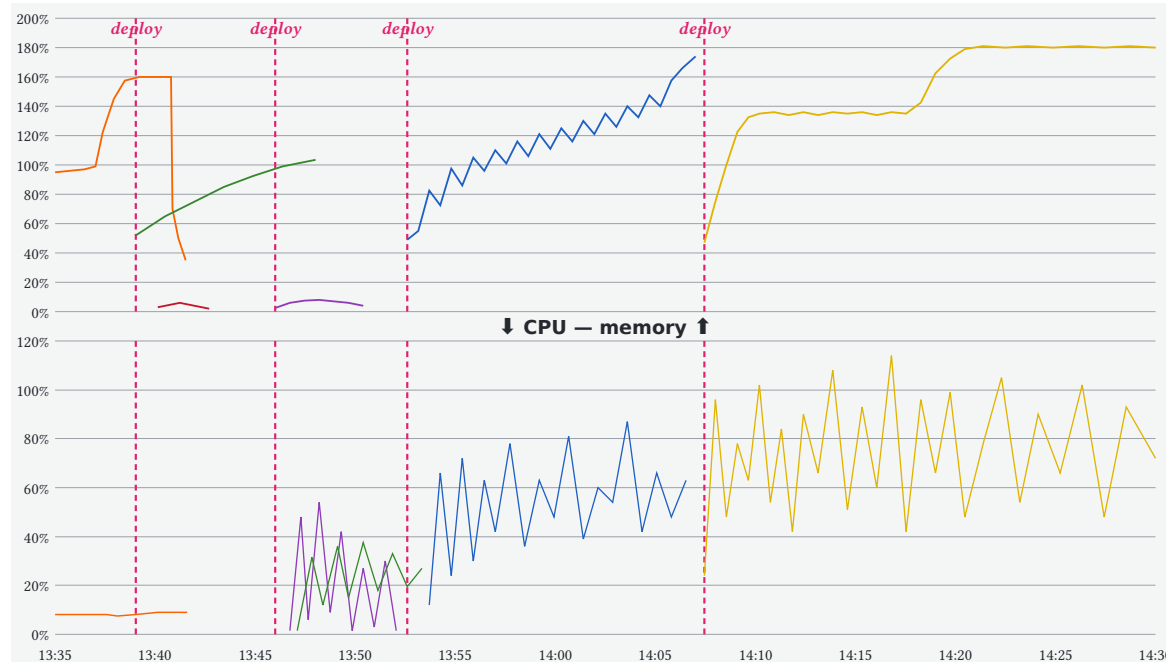
Så hva nå?





32+ GiB → **<1 GiB**
før d99f656 *etter*





Hver rød linje = ny deploy, ny hypotese. Kompilatoren fanger **syntaksen**; Grafana fanger **virkeligheten**.

Y-aksen: % av requested ressurs (ikke absolutt verdi).



Etterslep: kompilatorenn hjalp, og der den IKKE gjorde det

Kompilatoren som trygger

Bug	Hva kompilatoren sa	Hva som fanget det
Double-free <i>b2477a7 · 17:25</i>	OK — blindt over FFI-grensen	panic() + stacktrace i testmiljø
Silent data loss <i>4e2d9e3 · neste dag 15:41</i>	OK — syntaks riktig, semantikk feil	logging på ekte cluster
Neg cache + dedup <i>5760504 · 17:03</i>	OK (+ tester OK)	produksjonsdata bekreftet

Kompilatoren + typesystemet fanger mye. Feedback loop fanger resten.





Rydd opp først

clippy pedantic før AI slapp til



...

...



Fil-watch

kontinuerlig kompilering + tester i egen terminal



git add -p

manuelt syn på hver hunk før staging



Anti-juks-sjekk

agenten skal ikke fjerne tester for å få bygg grønt



Stor refaktor —

Per-gruppe BaseConsumer → delt AdminClient.

220 linjer unsafe FFI borte på én kveld.

Streaming pipeline vs batch jobbing → per-gruppe, maks N in-flight.

Stor optimalisering —

Logger viste `stream_metrics_ms()` = 93 % av tidsforbruket.

Cross-group dedup + negativ cache foreslått og implementert av AI.

Sentinel-jakt —

Metrikken viste 0 trass i reell lag.

Agenten sporet opp sentinel-verdi-tolkning i rust-rdkafka-forken.



Arbeidet hadde blitt gjort uansett — men ikke på helg.

Kort om meg	2
Motivasjon	6
Kompilatoren som trygger	11
FFI i fremmed land? Nix!	19
Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!	25
Avslutning	31



Tre problemer Rust ikke kunne fange

- librdkafka double-free ved shutdown
- Blocking FFI kvelte tokio — readiness-probe feilet
- SIGSEGV fra describe-call på 13 400+ consumer groups

Crashen skjer på C-bibliotekets egne tråder — usynlig for Rust.



JEMALLOC_OVERRIDE — én linje Nix

FFI i fremmed land? Nix!



- Minnet fragmenterte -> måtte bytte minneallokeringsverktøy.
- Rust-økosystemets vanlige vei bygget ikke for oss.

```
JEMALLOC_OVERRIDE = "${pkgs.jemalloc}/lib/libjemalloc.a";
```

Samme hack i en vanlig Dockerfile = kaos.





12:59

install SIGSEGV handler



14:49

Revert «install SIGSEGV handler»

~2 timer

«The signals are blocked in the C-level librdkafka library, rev [confluentinc/librdkafka#4571](https://github.com/confluentinc/librdkafka/pull/4571)»

Reproduserbart bygg + git = *turte å prøve, turte å revertere.*





Rydd opp først

clippy pedantic før AI slapp til



Begrenset handlingsrom

agenten får ikke committe, venter på klarsignal



Fil-watch

kontinuerlig kompilering + tester i egen terminal



git add -p

manuelt syn på hver hunk før staging



Anti-juks-sjekk

agenten skal ikke fjerne tester for å få bygg grønt

Minnefragmentering —

AI (1) forklarte situasjonen basert på bibliotekets kildekode, (2) foreslo ny minnehåndtering (`jemalloc`) som stabiliserte (3) minneforbruket og frigjøring av minne.

Nix-overstyring —

GCC bug i bundled `jemalloc` configure script. Agenten fant 1x linjes fiks vha. Nix.

Fryktløs utforskning —

Kræsje uten fornuftig feilkode/feilmelding. SIGSEGV-handler med `libc::backtrace` (1) ble installert, deretter (2) fjernet ~2t senere, når (3) det viste seg at `librdkafka` blokkerer signaler via `sigfillset()`. Agenten (4) fant allerede eksisterende issue #4571.



Uten Nix + git: ville aldri turt å prøve.



Kort om meg	2
Motivasjon	6
Kompilatoren som trygger	11
FFI i fremmed land? Nix!	19
Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!	25
Avslutning	31



confluentinc/librdkafka — 150 000 linjer C/C++, 251 C-filer, 14 år gammelt

- 10-talls commits bare siste 3 mnd pt. i dag
- Manuell memory management, ingen borrow checker, segfaults overalt

For en gjengs utvikler: **skremmende territorium.**

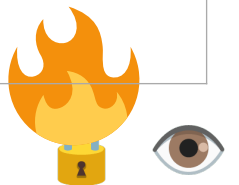


Feb 23–25: 72 timer FFI-helvete

Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!



Dato	Hva
23. feb 14:32	Fix double-free trigget av librdkafka ved shutdown
23. feb 15:08	jemalloc + pool metadata-refresh: -1
24. feb 10:52	block_in_place rundt blocking FFI
24. feb 12:59	Install SIGSEGV handler
24. feb 14:49	Revert — signals blocked i librdkafka
24. feb 14:58	Bound concurrent batches (100, 5-round chunks)
25. feb 10:19	Reduce timeout-thundering-herd





- Jeg **forket** et 14 år gammelt C-bibliotek
 - Jeg **patchet** det
 - Jeg **reverte** en feil-installert signal handler
 - Jeg **begrenset parallelitet** mot C-laget basert på faktisk SIGSEGV-logging

Jeg turte dette på grunn av verktøykjeden.

Rust + Nix + tester + CI + git = fryktløshet *per default*.





Rydd opp først

clippy pedantic før AI slapp til



Begrenset handlingsrom

agenten får ikke committe, venter på klarsignal



Fil-watch

kontinuerlig kompilering + tester i egen terminal



git add -p

manuelt syn på hver hunk før staging



Anti-juks-sjekk

agenten skal ikke fjerne tester for å få bygg grønt

Navigering i 150k linjer C —

Oppdagelse av fork —

Hypoteser som kunne fires raskt —



`sigfillset()` blokkerer alle signaler på `librdkafka`s broker-tråder. **Uten** agenten: *timer* med `grep` og `callgraph`-tracing.
Med: *minutter* til riktig fil.

Admin API manglet i `rust-rdkafka`. Agenten fant `j-santander/rust-rdkafka` som hadde C-bindings — rebased på master i vår fork.

Concurrency-grense mot C bibliotek: $5\text{batches} \times 10\text{ms}$ pause.
Agenten kommer med fiks basert på `librdkafka`-internals-forståelse. Fungerte på *første* forsøk.



14 år gammel C, ingen tidligere erfaring — likevel framover.

Kort om meg	2
Motivasjon	6
Kompilatoren som trygger	11
FFI i fremmed land? Nix!	19
Gå inn i 14 år gammelt C-bibliotek? Pft, barnemat!	25
Avslutning	31

Store refaktoreringer —

BaseConsumer → AdminClient, streaming pipeline, jemalloc som allocator. *Uten* agenten: sannsynligvis **ikke** påbegynt.



Arkeologi i ukjent terreng —

sigfillset() **innimellom** 150k linjer librdkafka. **Utvikle** Admin API i j-santander-forken. **Forstå** glibc-fragmenterings-mekanikken.

Veiviser, ikke ekspert — men det **holdt**.

Rotårsaks-analyse fra data —

93% av tidsbruk i timestamp-henting. Minne × consumer groups som dominerende allokering. Offset::End-sentinel som maskerte reell lag. Hypoteser jeg kunne utforske raskt.



**Sikringsteknikkene gjorde agenten trygg å bruke.
Agenten gjorde arbeidet mulig å fullføre.**

- Bolk 1** — Rusts kompilator + typesystem **fanger** mye av det LLM-en roter med. Tester & logging fanger resten. 
- Bolk 2** — Nix gjør reproduserbart bygg til **default**. JEMALLOC_OVERRIDE og patched rust-rdkafka blir én linje hver.
- Bolk 3** — Med feedback loops som holder, **tør** man å gå inn i *14 år gammel C-kode* — og **revertere** når det ikke virker.



Fryktløshet er en effekt av verktøykjeden.

GenAI blir ikke tryggere av seg selv med det første — det blir tryggere når noe fanger feilene den gjør.



Takk!



Christian Chavez — NAV IT / nais

Slides + notater: github.com/x10an14-nav/fryktlos-genai-koding

Case study:

nais/klag-exporter @ deploy_our_own_helmchart



Spørsmål?



- NAVs plattform er **hendelsesdrevet**
 - Livet Er En Strøm Av Hendelser (Leesah) filosofien
- Basert på Fred Georges «Rapids & Rivers» tankegang
 - Se også navikt/leesah-game
 - hendelsesdrevet applikasjonsutviklingsspill

*Konsekvens: **alt** blir Kafka-topics. Mange av dem.*